Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase II · Aprendizaje automático y redes neuronales

Módulo 10

Diferenciación automática

De la regla de la cadena a `loss.backward()`

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 10: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$f\colon \mathbb{R}^n \to \mathbb{R}^m$	función que se deriva, con n entradas y m salidas
$v_{1-n}, \dots, v_0$	variables de entrada de la traza de evaluación
$v_1, \dots, v_\ell$	variables intermedias, $v_i = \varphi_i(v_j)_{j \prec i}$
$j \prec i$	v_j es un argumento de la operación elemental φ_i
$\dot{v}_i$	tangente: derivada de v_i en una dirección fijada de las entradas
$\bar{v}_i = \partial y / \partial v_i$	adjunto de v_i respecto a la salida escalar y
$J_f(x) \in \mathbb{R}^{m \times n}$	jacobiano de f en x
$\mathbb{D}$, $a + b\varepsilon$	números duales, con $\varepsilon^2 = 0$
$\operatorname{ops}(f)$	número de operaciones aritméticas para evaluar f
N, D	número de parámetros de un modelo y de tokens de entrenamiento
$\mathbf{u}$	unidad de redondeo de la aritmética en coma flotante

Como en el módulo 9, los vectores que fluyen por una red se escriben como *filas* y las capas actúan por la derecha, $X \mapsto XW$; los gradientes respecto a una matriz tienen la forma de la matriz. En los temas 2 y 3, en cambio, el jacobiano actúa sobre vectores columna, $\dot{y} = J\dot{x}$, como en [2] y [10]. El código se escribe en Python y los nombres de PyTorch aparecen en `letra mecanográfica`.

Índice

Sobre este resumen **ii**

1 Grafos computacionales y métodos de diferenciación **1**
1.1 Trazas de evaluación . . . 1
1.2 Tres maneras de derivar . . . 2
1.2.1 Diferencias finitas . . . 2
1.2.2 Derivación simbólica . . . 2
1.2.3 Diferenciación automática . . . 2

2 El modo directo y los números duales **4**
2.1 Propagación de tangentes . . . 4
2.2 Números duales . . . 4
2.3 Coste del modo directo . . . 5

3 El modo inverso y la retropropagación **6**
3.1 Adjuntos . . . 6
3.2 El algoritmo . . . 7
3.3 El orden topológico . . . 7

4 Coste del gradiente y productos vector-jacobiano **8**
4.1 El principio del gradiente barato . . . 8
4.2 Asociatividad del producto de jacobianos . . . 8
4.3 Productos vector-jacobiano . . . 9
4.4 El cómputo de un paso de entrenamiento . . . 9

5 Memoria, segundo orden y método adjunto **11**
5.1 La memoria del modo inverso . . . 11
5.2 Recálculo por puntos de control . . . 11
5.3 Productos hessiana-vector . . . 12
5.4 Los adjuntos como multiplicadores de Lagrange . . . 12

6 Implementación: un autograd escalar y PyTorch **13**
6.1 Un autograd escalar . . . 13
6.2 PyTorch . . . 14
6.3 Acumulación de gradientes . . . 15

Bibliografía **16**

Tema 1

Grafos computacionales y métodos de diferenciación

Este tema fija el objeto que se deriva: un programa que calcula una función a partir de operaciones elementales. Se formaliza como traza de evaluación y como grafo dirigido acíclico, y se comparan las tres maneras de obtener derivadas con un ordenador: diferencias finitas, derivación simbólica y diferenciación automática. Se siguen las secciones 2 y 3 de [2], el capítulo 2 de [10], la §6.5.1 de [9] y las notas de la clase 8 de MIT 18.337 [17].

1.1 Trazas de evaluación

Todo programa que calcula un número real a partir de otros sin bifurcaciones que dependan de los datos ejecuta una sucesión finita de operaciones aritméticas y de funciones de biblioteca (exp, ln, sen, tanh…). Cuando hay bifurcaciones o bucles, cada ejecución concreta recorre una sucesión de este tipo, que depende de la entrada; las derivadas se toman respecto a esa sucesión.

Definición 1.1 (Operaciones elementales). Un conjunto de *operaciones elementales* es una colección finita Φ de funciones $\varphi\colon U_\varphi \subseteq \mathbb{R}^{k_\varphi} \to \mathbb{R}$, con U_φ abierto y $k_\varphi \le k$ argumentos, de clase C^1 y cuyas derivadas parciales $\partial\varphi/\partial u_j$ se pueden evaluar con un número acotado de operaciones. En este módulo Φ contiene la suma, la resta, el producto, la división, las potencias de exponente fijo, exp, ln, sen, cos y tanh.

Definición 1.2 (Traza de evaluación). Sea $f\colon \mathbb{R}^n \to \mathbb{R}^m$. Una *traza de evaluación* de f es una sucesión de variables

$$v_{1-n}, \dots, v_0,\ v_1, \dots, v_\ell$$

tal que $v_{i-n} = x_i$ para $i = 1, \dots, n$, cada variable intermedia se obtiene como $v_i = \varphi_i\big((v_j)_{j \prec i}\big)$ con $\varphi_i \in \Phi$ y $j \prec i \Rightarrow j < i$, y las salidas son $y_{m-i} = v_{\ell-i}$ para $i = 0, \dots, m-1$. La relación $j \prec i$ se lee «v_j es un argumento de φ_i».

La condición $j < i$ expresa que cada variable se calcula a partir de variables ya calculadas. Es la notación de Griewank y Walther [10, cap. 2], que Baydin *et al.* adoptan en [2, §3]: las entradas llevan índices no positivos, las intermedias, positivos, y las salidas son las últimas variables.

Definición 1.3 (Grafo computacional). El *grafo computacional* de una traza es el grafo dirigido $G = (V, E)$ con un nodo por variable y una arista $v_j \to v_i$ por cada par $j \prec i$. Los nodos sin aristas entrantes son las entradas.

Proposición 1.4. *El grafo computacional de una traza es acíclico, y el orden de la traza es un* orden topológico*: toda arista va de un índice menor a uno mayor.*

Observación 1.6. En un modelo de lenguaje, la traza de una pasada hacia delante tiene miles de millones de operaciones escalares, pero se organiza en unas pocas operaciones sobre tensores: productos de matrices, softmax, normalizaciones. El grafo que construyen los sistemas reales tiene un nodo por operación tensorial (tema 6); la teoría de los temas 2 a 5 se formula con escalares y se extiende sin cambios a bloques.

1.2 Tres maneras de derivar

1.2.1 Diferencias finitas

La aproximación más directa sustituye la derivada por un cociente incremental. Para $f\colon \mathbb{R}^n \to \mathbb{R}$ y la base canónica $e_1, \dots, e_n$,

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + he_i) - f(x)}{h} \qquad \text{o} \qquad \frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + he_i) - f(x - he_i)}{2h}.$$

En el módulo 4 (tema 6) se demostró que el error de truncamiento es $O(h)$ en la primera fórmula y $O(h^2)$ en la segunda, y que el redondeo añade un error de orden $\mathbf{u}|f|/h$, donde $\mathbf{u} \approx 1{,}1 \cdot 10^{-16}$ es la unidad de redondeo de doble precisión. El compromiso entre ambos da un paso óptimo $h^* \sim \mathbf{u}^{1/3}$ para la diferencia centrada, con un error mínimo de orden $\mathbf{u}^{2/3} \approx 10^{-11}$.

Proposición 1.7 (Coste de las diferencias finitas). *Aproximar el gradiente de $f\colon \mathbb{R}^n \to \mathbb{R}$ por diferencias centradas requiere $2n$ evaluaciones de f; por diferencias hacia delante, $n + 1$.*

Con $n = 10^{11}$ parámetros, ni la precisión ni el coste son aceptables. Las diferencias finitas se usan para *comprobar* un gradiente en unas pocas direcciones, nunca para entrenar [2, §2.1].

Observación 1.8 (Paso complejo). Si f es analítica y real sobre la recta real, el desarrollo de Taylor $f(x + ih) = f(x) + ihf'(x) - \frac{1}{2}h^2 f''(x) - \frac{i}{6}h^3 f'''(x) + \dots$ da

$$f'(x) = \frac{\operatorname{Im} f(x + ih)}{h} + O(h^2).$$

No hay resta de cantidades próximas, de modo que el error de redondeo no crece al disminuir h y puede tomarse $h = 10^{-20}$. Es un precursor del modo directo del tema 2: la unidad imaginaria hace el papel de un infinitesimal, aunque con $i^2 = -1$ en lugar de $\varepsilon^2 = 0$ [17].

1.2.2 Derivación simbólica

La derivación simbólica aplica las reglas del cálculo a la *expresión* de f y devuelve otra expresión. Es exacta, pero, si se aplica sin compartir subexpresiones, el tamaño del resultado puede crecer exponencialmente [2, §2.2].

1.2.3 Diferenciación automática

Definición 1.10 (Diferenciación automática). La *diferenciación automática* (*automatic differentiation*, AD) es la familia de técnicas que calculan valores numéricos de derivadas de una función definida por un programa, aplicando la regla de la cadena a los *valores* de la traza de evaluación y usando, en cada operación elemental, su derivada local conocida.

Como subraya [2, §3], la AD no manipula expresiones sino números: cada operación elemental aporta una derivada local que se combina con las demás. A diferencia de las diferencias finitas, el resultado es exacto salvo el redondeo de cada operación; a diferencia de la derivación simbólica, el coste es proporcional al de la traza. La regla de la cadena admite dos órdenes de aplicación, que dan los dos modos de la AD: el *directo* (*forward*), del tema 2, y el *inverso* (*reverse*), del tema 3.

Método	Exactitud	Coste de ∇f, $f\colon \mathbb{R}^n \to \mathbb{R}$	Uso
Diferencias finitas	error $O(h^2)$ más redondeo	$2n$ evaluaciones	comprobar
Simbólica	exacta	expresiones que pueden crecer	cálculo a mano
AD, modo directo	exacta salvo redondeo	n pasadas	pocas entradas
AD, modo inverso	exacta salvo redondeo	una pasada (teorema 4.1)	entrenar redes

Tema 2

El modo directo y los números duales

El modo directo propaga, junto con cada valor de la traza, su derivada respecto a una dirección fijada de las entradas. Este tema lo formula como propagación de tangentes, demuestra que calcula un producto jacobiano-vector y lo reinterpreta con los números duales, que reducen la regla de la cadena a aritmética. Se siguen la §3.1 de [2], incluida la §3.1.1 sobre números duales, el capítulo 3 de [10], las notas de la clase 8 de MIT 18.337 [17] y la clase 5 de MIT 18.S096 [18].

2.1 Propagación de tangentes

Se fija una traza de $f\colon \mathbb{R}^n \to \mathbb{R}^m$ (definición 1.2), un punto x y una dirección $\dot{x} \in \mathbb{R}^n$.

Definición 2.1 (Tangentes). Las *tangentes* de la traza son los números definidos por recurrencia como

$$\dot{v}_{i-n} = \dot{x}_i \quad (i = 1, \dots, n), \qquad \dot{v}_i = \sum_{j \prec i} \frac{\partial \varphi_i}{\partial v_j}\, \dot{v}_j \quad (i = 1, \dots, \ell),$$

donde las derivadas parciales se evalúan en los valores de la traza. El vector $\dot{x}$ se llama *semilla*.

Teorema 2.2 (El modo directo calcula $J\dot{x}$). *Sea $\gamma(t) = x + t\dot{x}$. Para cada i, $\dot{v}_i = \frac{d}{dt} v_i(\gamma(t))\big|_{t=0} = \nabla v_i(x)^\top \dot{x}$, donde $v_i(\cdot)$ denota la variable i como función de las entradas. En particular, el vector de tangentes de las salidas es $\dot{y} = J_f(x)\,\dot{x}$.*

Corolario 2.4 (Coste del jacobiano en modo directo). *Una pasada en modo directo da una columna de $J_f(x)$, la correspondiente a la semilla e_k. El jacobiano completo de $f\colon \mathbb{R}^n \to \mathbb{R}^m$ requiere n pasadas, independientemente de m.*

El corolario fija el campo de aplicación del modo directo: es óptimo cuando hay pocas entradas y muchas salidas y es inviable para el gradiente de una pérdida escalar respecto a 10^{11} parámetros, que exigiría 10^{11} pasadas.

2.2 Números duales

Definición 2.5 (Números duales). El conjunto de los *números duales* es $\mathbb{D} = \{a + b\varepsilon : a, b \in \mathbb{R}\}$, con la suma y el producto

$$(a + b\varepsilon) + (c + d\varepsilon) = (a + c) + (b + d)\varepsilon, \qquad (a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon.$$

El número a es la *parte real* y b, la *parte dual*.

El producto es el que resulta de multiplicar formalmente e imponer $\varepsilon^2 = 0$ con $\varepsilon \neq 0$. Con estas operaciones $\mathbb{D}$ es un anillo conmutativo con unidad, isomorfo a $\mathbb{R}[\varepsilon]/(\varepsilon^2)$, pero no es un cuerpo: $\varepsilon \cdot \varepsilon = 0$ con $\varepsilon \neq 0$, de modo que ε no tiene inverso [2, §3.1.1].

Proposición 2.6 (Inverso de un dual). *Si $a \neq 0$, el dual $a + b\varepsilon$ es invertible y*

$$\frac{1}{a + b\varepsilon} = \frac{1}{a} - \frac{b}{a^2}\,\varepsilon.$$

La parte dual del inverso es b veces la derivada de $u \mapsto 1/u$ en a, y la parte dual del producto es la regla de Leibniz. Esto no es casual, y motiva la definición siguiente.

Definición 2.7 (Extensión dual de una operación elemental). Sea $\varphi\colon U \subseteq \mathbb{R}^k \to \mathbb{R}$ de clase C^1. Su *extensión dual* es

$$\varphi(a_1 + b_1\varepsilon, \ldots, a_k + b_k\varepsilon) = \varphi(a) + \Big(\sum_{j=1}^{k} \frac{\partial \varphi}{\partial u_j}(a)\, b_j\Big)\varepsilon, \qquad a \in U.$$

Por ejemplo, $\exp(a + b\varepsilon) = e^a + e^a b\,\varepsilon$ y $\operatorname{sen}(a + b\varepsilon) = \operatorname{sen} a + (\cos a)\, b\,\varepsilon$.

Proposición 2.8 (Coherencia con la aritmética). *Para la suma, el producto y la inversión, la extensión dual de la definición 2.7 coincide con las operaciones de la definición 2.5 y de la proposición 2.6.*

Teorema 2.9 (Los duales propagan la derivada). *Sea $f\colon \mathbb{R}^n \to \mathbb{R}^m$ dada por una traza y evalúese la traza en aritmética dual, con entradas $x_k + \dot{x}_k\varepsilon$ y la extensión dual de cada operación elemental. Entonces cada variable vale $v_i + \dot{v}_i\varepsilon$, con $\dot{v}_i$ la tangente de la definición 2.1, y las salidas valen*

$$f(x + \dot{x}\,\varepsilon) = f(x) + J_f(x)\,\dot{x}\,\varepsilon.$$

El teorema justifica la lectura del desarrollo de Taylor del módulo 4 con un infinitesimal: $f(a + b\varepsilon) = f(a) + f'(a)b\varepsilon$ es el desarrollo de primer orden, y aquí el truncamiento es exacto porque los términos de orden superior contienen $\varepsilon^2 = 0$. Para evaluar $f'(a)$ basta con calcular $f(a + \varepsilon)$ y leer la parte dual.

Observación 2.11 (Duales con varias componentes). Si se toman p infinitesimales $\varepsilon_1, \ldots, \varepsilon_p$ con $\varepsilon_r\varepsilon_s = 0$ para todo r, s, cada variable lleva un vector de p partes duales, y con las semillas $x_k + \sum_r (\dot{x}^{(r)})_k \varepsilon_r$ una sola pasada calcula p productos $J\dot{x}^{(r)}$. Con $p = n$ y semillas $e_1, \ldots, e_n$ se obtiene el jacobiano entero en una pasada vectorial, cuyo coste sigue siendo proporcional a $n \cdot \operatorname{ops}(f)$ [17]. La biblioteca JAX ofrece esta operación como `jvp` (*Jacobian-vector product*).

2.3 Coste del modo directo

Proposición 2.12. *Existe una constante ω_{dir}, que depende solo del conjunto Φ de operaciones elementales, tal que una pasada en modo directo cuesta a lo sumo $\omega_{\mathrm{dir}} \cdot \operatorname{ops}(f)$ operaciones.*

Tema 3

El modo inverso y la retropropagación

El modo inverso recorre la traza dos veces: hacia delante, para calcular y guardar los valores, y hacia atrás, para propagar desde la salida la derivada de esta respecto a cada variable. Este tema define los adjuntos, demuestra que el algoritmo calcula un producto vector-jacobiano, estudia el papel del orden topológico y aplica el método a mano. Se siguen la §3.2 de [2], el capítulo 3 de [10], las §6.5.2–6.5.6 de [9], las notas «Backpropagation, Intuitions» de [5] y la §5.3 de [23].

3.1 Adjuntos

Se fija una traza de $f\colon \mathbb{R}^n \to \mathbb{R}$ con salida escalar $y = v_\ell$, y un punto x en el que se han calculado todos los valores v_i.

Definición 3.1 (Adjuntos). Los *adjuntos* de la traza se obtienen así: se inicializan $\bar{v}_\ell = 1$ y $\bar{v}_j = 0$ para $j < \ell$, y se recorren las operaciones en orden decreciente, $i = \ell, \ell-1, \dots, 1$, ejecutando para cada argumento $j \prec i$ la actualización

$$\bar{v}_j \;\mathrel{+}=\; \bar{v}_i \, \frac{\partial \varphi_i}{\partial v_j}.$$

El valor final de $\bar{v}_j$ se llama *adjunto* de v_j.

Equivalentemente, el valor final de cada adjunto es

$$\bar{v}_j = \sum_{i\,:\,j \prec i} \bar{v}_i \, \frac{\partial \varphi_i}{\partial v_j},$$

la suma sobre todas las operaciones que usan v_j. El símbolo $+=$ es esencial: si v_j alimenta a varias operaciones, su adjunto acumula una contribución de cada una.

Para enunciar qué calculan los adjuntos, sea $F_i(v_{1-n}, \dots, v_i)$ la salida que se obtiene al ejecutar las operaciones $i+1, \dots, \ell$ de la traza a partir de unos valores *arbitrarios* de las variables $v_{1-n}, \dots, v_i$, tratadas como independientes. Así, $F_\ell(v_{1-n}, \dots, v_\ell) = v_\ell$ y $F_0(x) = f(x)$, y $\partial F_i / \partial v_i$ mide la sensibilidad de la salida a una perturbación de v_i introducida justo después de calcularla.

Teorema 3.2 (Corrección del modo inverso). *Para cada $i \in \{0, \dots, \ell\}$, después de procesar las operaciones $\ell, \dots, i+1$, los acumuladores cumplen $\bar{v}_j = \partial F_i / \partial v_j$ para todo $j \le i$. En particular, al terminar,*

$$\bar{v}_{k-n} = \frac{\partial f}{\partial x_k}(x) \quad (k = 1, \dots, n), \qquad \textit{es decir,} \qquad (\bar{v}_{1-n}, \dots, \bar{v}_0) = \nabla f(x)^\top,$$

y el adjunto de cada variable intermedia es $\bar{v}_i = \partial F_i / \partial v_i$.

La demostración muestra por qué el orden importa: cuando se procesa la operación i, el acumulador $\bar{v}_i$ debe estar completo, lo que exige haber procesado antes todas las operaciones que usan v_i. Es la regla de la cadena en un grafo del módulo 9 (tema 5), ahora con su algoritmo.

Proposición 3.3 (El modo inverso calcula $\bar{y}^\top J$). *Sea $f\colon \mathbb{R}^n \to \mathbb{R}^m$ y $\bar{y} \in \mathbb{R}^m$. Si se inicializan los adjuntos de las salidas con las componentes de $\bar{y}$ y se ejecuta el recorrido de la definición 3.1, los adjuntos de las entradas forman el vector fila $\bar{x}^\top = \bar{y}^\top J_f(x)$. En consecuencia, el jacobiano completo requiere m pasadas inversas, una por fila.*

3.2 El algoritmo

Las derivadas locales usan valores de la pasada hacia delante: o en el nodo tanh, los factores en cada producto. Por eso el modo inverso debe *guardar* esos valores hasta la pasada hacia atrás; en un LLM, las activaciones guardadas ocupan a menudo más memoria que los pesos (tema 5).

3.3 El orden topológico

Definición 3.7 (Orden topológico). Un *orden topológico* de un grafo dirigido $G = (V, E)$ es una enumeración $u_1, \ldots, u_{|V|}$ de sus nodos tal que toda arista $u_r \to u_s$ cumple $r < s$.

El teorema 3.2 se ha enunciado con el orden de la traza, pero la demostración solo usa que cada operación se procesa después de todas las que usan su resultado. Por tanto, el modo inverso es correcto si las operaciones se procesan en el *inverso* de cualquier orden topológico del grafo.

Proposición 3.8 (Construcción por búsqueda en profundidad). *Sea G un grafo dirigido acíclico y sea el procedimiento* VISITAR(v)*: si v no está marcado, marcarlo, llamar a* VISITAR(u) *para cada predecesor u de v (cada u con $u \to v$) y, después, añadir v al final de una lista. Si se llama a* VISITAR *desde la salida, la lista resultante es un orden topológico del subgrafo de los nodos de los que depende la salida, y se construye en tiempo $O(|V| + |E|)$.*

Observación 3.10 (La cinta). Muchos sistemas no construyen el orden con la proposición 3.8, sino que registran las operaciones en una lista, la *cinta* (*tape* o lista de Wengert), en el orden en que se ejecutan. Ese orden es topológico, porque ninguna operación se ejecuta antes que sus argumentos, y basta recorrer la cinta al revés. Es el diseño de la tarea complementaria de autograd del curso 11-785 de Carnegie Mellon [4, §2.3], que sustituye el grafo por una lista lineal de operaciones.

Tema 4

Coste del gradiente y productos vector-jacobiano

Este tema responde a la pregunta que justifica el módulo: por qué el modo inverso calcula el gradiente de una función de millones de variables con un coste comparable al de evaluarla. Se demuestra el principio del gradiente barato, se compara el coste de los dos modos mediante la asociatividad del producto de jacobianos, se formulan las reglas locales de los sistemas reales como productos vector-jacobiano y se deduce de ellas la regla $C \approx 6ND$ del cómputo de entrenamiento. Se siguen el capítulo 4 de [10], la §3.2 de [2], la lección 6 de [12], el apartado «Gradients for vectorized operations» de [5] y el artículo de Kaplan *et al.* [13].

4.1 El principio del gradiente barato

Teorema 4.1 (Gradiente barato). *Sea $f\colon \mathbb{R}^n \to \mathbb{R}$ dada por una traza con operaciones del conjunto Φ de la definición 1.1, cada una con a lo sumo k argumentos. Existe una constante ω, que depende solo de Φ y no de n ni de la traza, tal que el modo inverso calcula $\nabla f(x)$ con a lo sumo $\omega \cdot \operatorname{ops}(f)$ operaciones.*

La constante de la demostración es pesimista. Un análisis más fino de cada operación da cotas pequeñas: Baydin *et al.*, siguiendo a Griewank y Walther, indican que el cociente entre el coste del modo inverso y el de la función está por debajo de 6 y suele estar entre 2 y 3 [2, §3.2], [10, cap. 4]. Para funciones racionales, un resultado de este tipo fue demostrado por Baur y Strassen [1]. Lo esencial es que la cota no depende de n: diez derivadas y 10^{11} derivadas cuestan, en relación con f, lo mismo.

Corolario 4.2 (Elección del modo). *Para $f\colon \mathbb{R}^n \to \mathbb{R}^m$, el jacobiano completo cuesta del orden de $n \cdot \operatorname{ops}(f)$ en modo directo y de $m \cdot \operatorname{ops}(f)$ en modo inverso. Conviene el directo si $n \ll m$ y el inverso si $m \ll n$. Entrenar un LLM corresponde a $n \approx 10^{11}$ y $m = 1$.*

El precio del modo inverso es la memoria: la pasada hacia atrás necesita los valores de la pasada hacia delante. En el modo directo, en cambio, cada tangente se descarta en cuanto deja de usarse. El tema 5 estudia este compromiso.

4.2 Asociatividad del producto de jacobianos

Sea $f = f_L \circ \cdots \circ f_1$ una composición de funciones diferenciables. Por la regla de la cadena (módulo 4), $J_f = J_{f_L} \cdots J_{f_1}$, con cada jacobiano evaluado en el punto correspondiente. El producto de matrices es asociativo y el resultado no depende del orden en que se agrupen los factores, pero el coste sí.

Proposición 4.3 (Coste de las dos agrupaciones). *Sea* $f\colon \mathbb{R}^m \to \mathbb{R}$ *con* $J_f = J_L J_{L-1} \cdots J_1$, *donde* $J_L \in \mathbb{R}^{1\times m}$ *y* $J_1, \dots, J_{L-1} \in \mathbb{R}^{m\times m}$, *con* $L \geq 2$, *y cuéntese como coste de un producto* $(p \times q)(q \times r)$ *el número* pqr *de multiplicaciones escalares. Entonces:*

(i) agrupando de derecha a izquierda, $J_L\big(J_{L-1}(\cdots(J_2 J_1))\big)$, *el coste es* $(L-2)m^3 + m^2$;

(ii) agrupando de izquierda a derecha, $\big(((J_L J_{L-1})J_{L-2})\cdots\big)J_1$, *el coste es* $(L-1)m^2$.

La agrupación (i) es la del modo directo con todas las semillas a la vez: propaga la matriz $\partial v/\partial x$, de tamaño $m \times m$. La agrupación (ii) es la del modo inverso: lo que viaja hacia atrás es siempre un vector fila, el adjunto. Con $L = 3$ y $m = 100$, el cociente de costes es $(100^3 + 100^2)/(2 \cdot 100^2) = 50{,}5$, y en general crece linealmente con m. Para composiciones que no son cadenas, encontrar la agrupación óptima es un problema difícil, estudiado en [10, cap. 9 y 10].

4.3 Productos vector-jacobiano

Los sistemas reales no trabajan con escalares, sino con operaciones sobre tensores, y nunca construyen sus jacobianos [12].

Definición 4.4 (Producto vector-jacobiano). Sea $g\colon \mathbb{R}^p \to \mathbb{R}^q$ una operación con entrada u y salida $w = g(u)$, y sea $\bar{w} \in \mathbb{R}^{1\times q}$ el adjunto de la salida. El *producto vector-jacobiano* (VJP) de g en u es la aplicación lineal $\bar{w} \mapsto \bar{u} = \bar{w}\, J_g(u) \in \mathbb{R}^{1\times p}$.

Por el teorema 3.2, aplicado por bloques, el modo inverso sobre un grafo de operaciones tensoriales solo necesita, para cada tipo de operación, una función que calcule su VJP. Si la entrada o la salida son matrices, se aplanan para la definición y el resultado se devuelve con la forma original.

Proposición 4.6 (VJP de la capa lineal). *Sea* $Y = XW$ *con* $X \in \mathbb{R}^{B\times n}$, $W \in \mathbb{R}^{n\times m}$, *y sea* $\bar{Y} \in \mathbb{R}^{B\times m}$ *el adjunto de la salida. Entonces*

$$\bar{X} = \bar{Y}\, W^\top \in \mathbb{R}^{B\times n}, \qquad \bar{W} = X^\top \bar{Y} \in \mathbb{R}^{n\times m}.$$

Proposición 4.8 (VJP de la softmax). *Sea* $p = \mathrm{softmax}(z)$ *con* $z \in \mathbb{R}^{1\times K}$ *y sea* $\bar{p} \in \mathbb{R}^{1\times K}$. *Entonces*

$$\bar{z} = \bar{p}\, J_{\mathrm{softmax}}(z) = p \odot \big(\bar{p} - \langle \bar{p}, p\rangle\, \mathbf{1}\big),$$

donde $\langle \bar{p}, p\rangle = \sum_k \bar{p}_k p_k$ *y* $\mathbf{1} = (1, \dots, 1)$. *El cálculo cuesta* $O(K)$ *operaciones.*

Corolario 4.9 (Softmax con entropía cruzada). *Si* $L = -\ln p_y$, *entonces* $\bar{z} = p - e_y$.

En la práctica, la softmax y la entropía cruzada se tratan como una sola operación con VJP $p - e_y$, que además es numéricamente estable: es lo que hace `F.cross_entropy` en PyTorch, que recibe logits y no probabilidades (tema 6).

4.4 El cómputo de un paso de entrenamiento

Se cuenta como FLOP una suma o un producto en coma flotante. Un producto de matrices $(p \times q)(q \times r)$ cuesta $pr(2q-1) \approx 2pqr$ FLOPs.

Proposición 4.10 (Coste de una capa lineal). *Para $Y = XW$ con $X \in \mathbb{R}^{B\times n}$ y $W \in \mathbb{R}^{n\times m}$, la pasada hacia delante cuesta unos $2Bnm$ FLOPs y la pasada hacia atrás unos $4Bnm$: $2Bnm$ para $\bar{X}$ y $2Bnm$ para $\bar{W}$. El total es $6Bnm = 6 \cdot (\text{parámetros}) \cdot (\text{filas del lote})$.*

Corolario 4.11 (Regla $6ND$). *Si el cómputo de un modelo con N parámetros está dominado por productos de matrices en los que cada parámetro actúa una vez por token, entrenarlo con D tokens cuesta $C \approx 6ND$ FLOPs: $2ND$ en las pasadas hacia delante y $4ND$ en las pasadas hacia atrás.*

La aproximación ignora las operaciones que no son productos por pesos, como el producto de la atención entre consultas y claves, que crece con la longitud del contexto, y el hecho de que la primera capa no necesita $\bar{X}$. Kaplan *et al.* la usan como base de las leyes de escalado [13], que se estudian en el módulo 16. Por ejemplo, un modelo de $N = 10^8$ parámetros entrenado con $D = 2 \cdot 10^9$ tokens requiere $C \approx 6 \cdot 10^8 \cdot 2 \cdot 10^9 = 1{,}2 \cdot 10^{18}$ FLOPs; a 10^{14} FLOP/s efectivos, unos $1{,}2 \cdot 10^4$ s, algo más de tres horas.

Tema 5

Memoria, segundo orden y método adjunto

El modo inverso compra su eficiencia en operaciones con memoria: necesita los valores de la pasada hacia delante. Este tema cuantifica esa memoria y estudia el recálculo por puntos de control, que la reduce a cambio de cómputo; obtiene productos hessiana-vector combinando los dos modos, y muestra que los adjuntos son multiplicadores de Lagrange, lo que conecta la retropropagación con el método adjunto del control óptimo y de la física. Se siguen los capítulos 4 y 12 de [10], el artículo de Chen *et al.* [3], el de Pearlmutter [20], la §3.3 de [2] y la tarea 2 de CS336 [7].

5.1 La memoria del modo inverso

Proposición 5.1 (Memoria de una cadena de capas). *Sea $f = f_L \circ \cdots \circ f_1$ una red de L capas cuyas activaciones intermedias $h_k = f_k(h_{k-1})$ ocupan a unidades de memoria cada una, y supóngase que el VJP de la capa k necesita h_{k-1}. Evaluar f requiere memoria $O(a)$, mientras que el modo inverso de la figura 3.1 requiere guardar $h_0, \ldots, h_{L-1}$, es decir, memoria La.*

En un LLM, a es proporcional al número de tokens del lote por la dimensión del modelo, y la memoria de las activaciones supera a menudo la de los pesos (módulo 17).

5.2 Recálculo por puntos de control

La idea del recálculo (*gradient checkpointing* o *rematerialization*) consiste en guardar solo algunas activaciones, los *puntos de control*, y recalcular las demás cuando la pasada hacia atrás las necesita [3], [10, cap. 12].

Proposición 5.2 (Un nivel de puntos de control). *En la situación de la proposición 5.1, divídanse las L capas en L/k tramos de k capas consecutivas y guárdese durante la pasada hacia delante solo la entrada de cada tramo. En la pasada hacia atrás, al llegar a cada tramo se repite su pasada hacia delante desde el punto de control, guardando sus k activaciones, y se retropropaga por él. Entonces:*

(i) la memoria de activaciones es $M(k) = (L/k + k)\,a$, mínima para $k = \sqrt{L}$, con valor $2\sqrt{L}\,a$;

(ii) el cómputo adicional es una pasada hacia delante más.

Con las cuentas de la proposición 4.10, una pasada hacia delante extra cuesta $2N$ FLOPs por token, y el paso de entrenamiento pasa de $6N$ a $8N$: un 33 % más de cómputo a cambio de

reducir la memoria de L a $2\sqrt{L}$ activaciones. Aplicando la idea de forma recursiva, la memoria baja a $O(\log L)$ con un cómputo de $O(L \log L)$ evaluaciones de capa [3]; Griewank y Walther estudian los esquemas óptimos de este tipo [10, cap. 12].

5.3 Productos hessiana-vector

Las derivadas de segundo orden se obtienen aplicando la AD a un programa que, a su vez, calcula derivadas.

Proposición 5.4 (Directo sobre inverso). *Sea $f\colon \mathbb{R}^n \to \mathbb{R}$ de clase C^2 dada por una traza cuyas operaciones elementales son de clase C^2, y sean $x, v \in \mathbb{R}^n$. Si se ejecuta el modo inverso (que calcula ∇f) en aritmética dual, con entradas $x + v\varepsilon$, la parte dual del resultado es el producto hessiana-vector $H_f(x)\,v$. Su coste es a lo sumo una constante, independiente de n, por* $\operatorname{ops}(f)$.

La técnica es la de Pearlmutter [20] y permite usar información de curvatura sin formar el hessiano, que para $n = 10^9$ tendría 10^{18} entradas. El hessiano completo requiere n productos hessiana-vector, uno por columna, y cuesta del orden de $n \cdot \operatorname{ops}(f)$, frente al $O(\operatorname{ops}(f))$ del gradiente.

5.4 Los adjuntos como multiplicadores de Lagrange

Teorema 5.6 (Método adjunto). *Sea una traza de $f\colon \mathbb{R}^n \to \mathbb{R}$ y considérese la lagrangiana*

$$\mathcal{L}(x, v_1, \dots, v_\ell, \lambda_1, \dots, \lambda_\ell) = v_\ell - \sum_{i=1}^{\ell} \lambda_i \Big(v_i - \varphi_i\big((v_j)_{j \prec i}\big) \Big),$$

donde $v_{k-n} = x_k$. Si los v_i toman los valores de la traza y los multiplicadores se eligen de modo que $\partial \mathcal{L}/\partial v_i = 0$ para $i = 1, \dots, \ell$, entonces $\lambda_i = \bar{v}_i$ para todo i y $\nabla f(x) = \nabla_x \mathcal{L}$.

El ejemplo es la retropropagación a través del tiempo de una red recurrente: un mismo parámetro actúa en cada paso y su gradiente es la suma de las contribuciones de todos ellos. En control óptimo, en diseño aerodinámico y en la asimilación de datos meteorológicos (el método 4D-Var), el mismo esquema se aplica a ecuaciones diferenciales, integrando una ecuación adjunta hacia atrás en el tiempo para obtener el gradiente de un coste respecto a millones de condiciones iniciales o parámetros.

Observación 5.8 (Historia). Linnainmaa describió el modo inverso en 1970 como herramienta para analizar errores de redondeo acumulados [15]; Rumelhart, Hinton y Williams lo popularizaron en 1986 para entrenar redes neuronales [22]. Baydin *et al.* reconstruyen esta historia y sus redescubrimientos [2, §3.3].

Tema 6

Implementación: un autograd escalar y PyTorch

Todo lo anterior cabe en unas decenas de líneas de Python. Este tema describe un sistema de diferenciación automática en modo inverso que opera con escalares, como micrograd [14], demuestra que basta un puñado de reglas locales, explica por qué los sistemas reales trabajan con tensores y presenta las piezas de PyTorch que se usan en el resto del curso: tensores con `requires_grad`, `backward`, `nn.Module` y los optimizadores. Se siguen el artículo de Paszke *et al.* [19], la nota «Autograd mechanics» de la documentación de PyTorch [21], la §2.5 de [23] y la tarea complementaria de autograd del curso 11-785 de Carnegie Mellon [4].

6.1 Un autograd escalar

Definición 6.1 (Nodo de un autograd escalar). Un *nodo* es un objeto con cuatro atributos: su valor `dato`; su adjunto `grad`, inicializado a 0; la lista `_prev` de los nodos de los que se ha calculado, y una función `_backward` que, leída la variable `grad` del propio nodo, suma a la `grad` de cada nodo de `_prev` su contribución según la regla local. Cada operación entre nodos crea un nodo nuevo y le asigna su `_backward`.

Operación	Hacia delante	Regla local (`_backward`)
$s = a + b$	$a + b$	$\bar{a}$ += $\bar{s}$; $\bar{b}$ += $\bar{s}$
$p = a \cdot b$	$a \cdot b$	$\bar{a}$ += $b\,\bar{p}$; $\bar{b}$ += $a\,\bar{p}$
$q = a^k$, k fijo	a^k	$\bar{a}$ += $k\,a^{k-1}\,\bar{q}$
$t = \tanh a$	$\tanh a$	$\bar{a}$ += $(1 - t^2)\,\bar{t}$
$e = \exp a$	e^a	$\bar{a}$ += $e\,\bar{e}$
$l = \ln a$	$\ln a$	$\bar{a}$ += $\bar{l}/a$

El método `backward()` del nodo de salida construye un orden topológico con la proposición 3.8, pone su propio `grad` a 1 y llama a `_backward` de cada nodo en orden inverso. Por el teorema 3.2, al terminar cada `grad` contiene la derivada de la salida respecto a ese nodo. El núcleo del sistema se muestra en la figura 6.1.

Las demás operaciones de la tabla se escriben como `__mul__`. Si una misma variable aparece en los dos factores, como en `x * x`, las dos líneas de `_backward` suman cada una su contribución a `x.grad`, y el resultado es $2x$, como exige la acumulación del tema 3.

Proposición 6.2 (Operaciones derivadas). *Si la resta, la negación y la división se definen como $-a = (-1)\cdot a$, $a - b = a + (-1)\cdot b$ y $a/b = a \cdot b^{-1}$, el sistema calcula correctamente sus derivadas sin reglas propias. Por ejemplo, para $f = a/b$ se obtiene $\bar{a} = 1/b$ y $\bar{b} = -a/b^2$.*

Con estos nodos se construye una red: una neurona calcula $\tanh\big(\sum_i w_i x_i + b\big)$; una capa es una lista de neuronas, y un perceptrón multicapa, una lista de capas. Un perceptrón $3 \to$

$4 \to 4 \to 1$ tiene $(3\cdot 4+4)+(4\cdot 4+4)+(4\cdot 1+1) = 41$ parámetros. Cada paso de descenso de gradiente tiene tres fases: calcular la pérdida; poner a cero todos los `grad` y llamar a `backward`; y actualizar cada parámetro con $p \leftarrow p - \eta\,\bar{p}$. Si se omite la puesta a cero, los adjuntos de pasos sucesivos se suman por la acumulación de la definición 3.1.

Observación 6.4 (Límite del enfoque escalar). Un producto de dos matrices 1000×1000 ejecuta 10^9 multiplicaciones y casi otras tantas sumas: un autograd escalar crearía del orden de $2\cdot 10^9$ nodos, cada uno con su objeto de Python. Los sistemas reales usan *tensores*: un nodo por operación sobre arrays enteros, con un VJP vectorizado como los de la proposición 4.6 y la proposición 4.8, que se ejecuta en una GPU. La tarea complementaria del curso 11-785 [4, §1 y §3.2] subraya este cambio de granularidad: el autograd trabaja a nivel de operación, no de capa, y cada operación necesita su función hacia atrás.

6.2 PyTorch

PyTorch [19] es, en esencia, el sistema anterior con tensores, ejecución en GPU y cientos de operaciones con su VJP. Es la herramienta del curso a partir de este módulo. El código de referencia publicado para los modelos LLaMA de Meta usa PyTorch [16]; el informe técnico de Gemini indica que se entrenó con JAX y Pathways [8], y el código de los pesos abiertos de Grok-1, un modelo de 314 000 millones de parámetros, también es de JAX [11]. Anthropic no ha publicado los detalles de su pila de software para Claude.

Pieza de PyTorch	Función	Equivalente escalar
`Tensor` con `requires_grad`	array que registra sus operaciones	`Valor`
`grad_fn`, `backward()`	nodo del grafo y modo inverso	`_backward`, `backward()`
`nn.Module`	agrupa parámetros, define `forward`	neurona, capa, MLP
`torch.optim` (SGD, AdamW)	aplica la actualización	$p \leftarrow p - \eta\,\bar{p}$

La nota «Autograd mechanics» de la documentación [21] precisa varios comportamientos que se siguen de la teoría de los temas anteriores.

(a) *Grafo dinámico.* El grafo se reconstruye desde cero en cada iteración, al ejecutar el código; por eso se puede usar el control de flujo de Python, y las derivadas son las de la traza ejecutada (tema 1).

(b) *Acumulación.* Durante `backward()`, solo los tensores *hoja* (los creados por el usuario) con `requires_grad=True` reciben gradiente, y este se *suma* a su atributo `.grad`. Por eso cada paso empieza con `opt.zero_grad()`.

(c) *Tensores guardados.* Cada operación guarda los tensores que necesita su VJP. Tras la pasada hacia atrás, esos tensores se liberan; una segunda llamada a `backward()` sobre el mismo grafo produce un error, salvo que la primera se haya hecho con `retain_graph=True`.

(d) *Desactivar el registro.* Dentro de `with torch.no_grad():` las operaciones no crean nodos ni guardan tensores; se usa para evaluar en validación y para actualizar los parámetros sin registrar la actualización.

Observación 6.6 (Dos detalles de convención). `nn.Linear(n, m)` guarda su peso con forma (m, n) y calcula $XW^\top + b$, de modo que `weight` es la traspuesta de la W del módulo 9. Y `F.cross_entropy` recibe logits, no probabilidades: aplica internamente una log-softmax estable (módulo 11) y su VJP es el del corolario 4.9.

6.3 Acumulación de gradientes

La acumulación en `.grad`, que obliga a llamar a `zero_grad()`, es también una herramienta.

Proposición 6.7 (Microlotes). *Sea* $\mathcal{L}(\theta) = \frac{1}{|S|}\sum_{s\in S}\ell_s(\theta)$ *la pérdida media sobre un lote* S*, partido en* k *microlotes disjuntos* $S_1, \dots, S_k$ *del mismo tamaño. Si, sin poner a cero los gradientes entre ellos, se llama a* `backward()` *sobre* $\mathcal{L}_r/k$ *para cada* r*, con* $\mathcal{L}_r = \frac{1}{|S_r|}\sum_{s\in S_r}\ell_s$*, el gradiente acumulado es* $\nabla\mathcal{L}(\theta)$*.*

Así se entrenan los LLM con lotes de millones de tokens que no caben a la vez en la memoria de una GPU: se procesan microlotes sucesivos, se acumulan sus gradientes y se da un solo paso del optimizador. Si los microlotes tuvieran tamaños distintos, habría que ponderar cada $\mathcal{L}_r$ por $|S_r|/|S|$ en lugar de por $1/k$.

Bibliografía

[1] W. Baur y V. Strassen. The complexity of partial derivatives. *Theoretical Computer Science*, 22(3):317–330, 1983.

[2] A. G. Baydin, B. A. Pearlmutter, A. A. Radul y J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018. https://jmlr.org/papers/v18/17-468.html

[3] T. Chen, B. Xu, C. Zhang y C. Guestrin. Training deep nets with sublinear memory cost. arXiv:1604.06174, 2016. https://arxiv.org/abs/1604.06174

[4] B. Raj y equipo docente de 11-785. 11-785: Introduction to Deep Learning. Tarea 1, parte complementaria, «An Introduction to Autograd». Carnegie Mellon University, otoño de 2021. https://github.com/CMU-IDeeL/new_grad

[5] A. Karpathy y equipo docente de CS231n. CS231n: Deep Learning for Computer Vision. Notas del curso, «Backpropagation, Intuitions». Stanford University. https://cs231n.github.io/optimization-2/

[6] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 1, «Basics», versión 26.0.3. Stanford University, primavera de 2026. https://github.com/stanford-cs336/assignment1-basics

[7] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 2, «Systems», versión 26.1.3. Stanford University, primavera de 2026. https://github.com/stanford-cs336/assignment2-systems

[8] Gemini Team, Google. Gemini: a family of highly capable multimodal models. arXiv:2312.11805, 2023. https://arxiv.org/abs/2312.11805

[9] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. §6.5, «Back-Propagation and Other Differentiation Algorithms». https://www.deeplearningbook.org/

[10] A. Griewank y A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2.ª ed. SIAM, 2008.

[11] xAI. Grok-1: código de ejemplo en JAX para cargar y ejecutar el modelo de pesos abiertos. 2024. https://github.com/xai-org/grok-1

[12] R. Grosse. Lecture 6: Automatic Differentiation. Notas del curso CSC421/2516, «Neural Networks and Deep Learning». University of Toronto, 2019. https://www.cs.toronto.edu/~rgrosse/courses/csc421_2019/

[13] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu y D. Amodei. Scaling laws for neural language models. arXiv:2001.08361, 2020. https://arxiv.org/abs/2001.08361

[14] A. Karpathy. micrograd: un pequeño motor de diferenciación automática en modo inverso sobre un grafo construido dinámicamente. 2020. https://github.com/karpathy/micrograd

[15] S. Linnainmaa. The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors. Tesis de maestría, Universidad de Helsinki, 1970.

[16] Meta. Llama 2: código de inferencia de referencia (PyTorch). 2023. https://github.com/meta-llama/llama

[17] C. Rackauckas. Parallel Computing and Scientific Machine Learning (18.337). Notas de la clase 8, «Forward-Mode Automatic Differentiation (AD) via High Dimensional Algebras». Massachusetts Institute of Technology, 2020. https://book.sciml.ai/

[18] A. Edelman y S. G. Johnson. 18.S096: Matrix Calculus for Machine Learning and Beyond. Clase 5, partes 2 y 3. Massachusetts Institute of Technology, MIT OpenCourseWare, IAP de enero de 2023. https://ocw.mit.edu/courses/18-s096-matrix-calculus-for-machine-learning-and-beyond-january-iap-2023/

[19] A. Paszke *et al.* PyTorch: an imperative style, high-performance deep learning library. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. https://arxiv.org/abs/1912.01703

[20] B. A. Pearlmutter. Fast exact multiplication by the Hessian. *Neural Computation*, 6(1):147–160, 1994.

[21] PyTorch. Autograd mechanics. Documentación de PyTorch, versión 2.14. https://docs.pytorch.org/docs/stable/notes/autograd.html

[22] D. E. Rumelhart, G. E. Hinton y R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[23] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. §2.5, «Automatic Differentiation», y §5.3, «Forward Propagation, Backward Propagation, and Computational Graphs». https://d2l.ai/